

# Raport Proiect: Gestiunea Tranzacțiilor și Concurenței

## Laborator 2 – Sisteme de Gestiune a Bazelor de Date

**Nume:** Acul Mathyas

**Data:** 16.03.2026

### 1. Explicația fiecărei probleme de concurență demonstrate

Pentru a evidenția problemele care apar atunci când mai multe tranzacții accesează simultan aceleași date, am implementat fire de execuție paralele (threads) care simulează comportamentul unui sistem multi-user.

- **1.1. Dirty Read**

- **Explicație:** Apare atunci când o tranzacție (T2) citește date care au fost modificate de o altă tranzacție (T1), dar care nu au fost încă salvate definitiv (COMMIT). Dacă T1 face ROLLBACK, datele citite de T2 nu au existat niciodată oficial. În implementare, Thread B citește un salariu modificat temporar de Thread A, înainte de anularea tranzacției.

```
> Task :org.example.Main.main()
Running with possibility of dirty reads:
Thread A: Updated balance, but not committed yet.
Thread B: Read salary = 5000
Thread B: Committed transaction.
Thread A: Rolled back transaction. Balance resets to original value.

Now running with prevention of dirty reads:
Thread A: Updated balance, but not committed yet.
Thread A: Rolled back transaction. Balance resets to original value.
Thread B: Read salary = 6000
Thread B: Committed transaction.
-----
```

- **1.2. Non-Repeatable Read**

- **Explicație:** Se manifestă când o tranzacție citește același rând de două ori și obține valori diferite. Acest lucru se întâmplă deoarece o altă tranzacție a intervenit și a dat UPDATE pe acel rând între cele două citiri.

```

> Task :org.example.Main.main()
Running with possibility of non-repeatable reads:|
Thread A: First read salary = 5000
Thread B: Updated salary to 4000.
Thread B: Committed transaction.
Thread A: Second read salary = 4000
Thread A: Committed transaction.

Now running with prevention of non-repeatable reads:
Thread A: First read salary = 5000
Thread A: Second read salary = 5000
Thread A: Committed transaction.
Thread B: Updated salary to 4000.
Thread B: Committed transaction.
-----

```

- **1.3. Phantom Read**

- **Explicație:** Spre deosebire de problema anterioară care afectează un rând existent, "Phantom Read" apare la interogările pe seturi de rânduri (ex: clauza WHERE). O tranzacție numără angajații, iar între timp o altă tranzacție face un INSERT care corespunde criteriilor. La a doua numărătoare, apare un rând "fantomă" în plus.

```

> Task :org.example.Main.main()
Running with possibility of phantom reads:
Thread A: First read employee count in department 5 = 0
Thread B: Inserted new employee into department 5.
Thread B: Committed transaction.
Thread A: Second read employee count in department 5 = 1
Thread A: Committed transaction.

Now running with prevention of phantom reads:
Thread A: First read employee count in department 5 = 0
Thread A: Second read employee count in department 5 = 0
Thread A: Committed transaction.
Thread B: Inserted new employee into department 5.
Thread B: Committed transaction.
-----

```

- **1.4. Lost Update**

- **Explicație:** Are loc atunci când două tranzacții citesc concomitent o valoare (ex: salariul = 5000), fac calcule separate în aplicație și apoi o suprascriu. Prima suprascriere este ștearsă (pierdută) de cea de-a doua. În codul meu, Thread A a suprascris munca lui Thread B.

```
> Task :org.example.Main.main()
Running with possibility of lost updates:
Thread A: Read salary = 5000
Thread B: Read salary = 5000
Thread B: Updated salary to 4500 and committed transaction.
Thread A: Updated salary to 6000
Thread A: Committed transaction.
Final salary after both transactions: 6000

Now running with prevention of lost updates:
Thread A: Read salary = 5000
Thread B: Read salary = 5000
Thread B: Updated salary to 4500 and committed transaction.
Thread A was interrupted: Transaction (Process ID 89) was deadlocked on lock resources with another process
Final salary after both transactions: 4500
-----
```

- **1.5. Deadlock**

- **Explicație:** Un blocaj circular în care Thread A a blocat Angajatul 1 și așteaptă să acceseze Angajatul 2, în timp ce Thread B a blocat Angajatul 2 și îl așteaptă pe 1. Sistemul SGBD este forțat să intervină și să oprească una dintre tranzacții ("Deadlock Victim").

```
> Task :org.example.Main.main()
Running with possibility of deadlock:
Thread A: Starting transaction and locking employee 1
Thread B: Starting transaction and locking employee 2
Thread A: Attempting to lock employee 2
Thread B: Attempting to lock employee 1
Thread B: Committed transaction.
Thread A was interrupted: Transaction (Process ID 88) was deadlocked on lock resources with another process

Now running with prevention of deadlock:
Thread A: Starting transaction and locking employee 1
Thread B: Starting transaction and locking employee 1
Thread A: Attempting to lock employee 2
Thread A: Committed transaction.
Thread B: Attempting to lock employee 2
Thread B: Committed transaction.
-----
```

## 2. Analiza nivelurilor de izolare: Când să folosiți fiecare și Scenarii din lumea reală

Soluția pentru problemele demonstrate la secțiunea anterioară o reprezintă nivelurile de izolare, care impun lacăte (locks) pe date.

- **READ UNCOMMITTED**

- **Când se folosește:** Când viteza și concurența sunt absolut prioritare, iar precizia matematică la nivel de rând nu este necesară. Nu folosește lacăte de citire.
- **Scenariu din lumea reală:** Generarea de rapoarte statistice pe volume uriașe de date (ex: calcularea mediei de vârstă a milioane de utilizatori de pe o rețea socială). O abatere minoră cauzată de un "Dirty Read" nu afectează rezultatul final, dar oferă un spor uriaș de viteză sistemului.

- **READ COMMITTED**

- **Când se folosește:** Este setarea implicită în majoritatea bazelor de date. Se folosește când vrem să evităm datele "murdare", dar dorim să menținem o concurență ridicată. Eliberează lacătul imediat după citire.
- **Scenariu din lumea reală:** Aplicații web uzuale, cum ar fi un magazin online (vizualizarea catalogului de produse) sau verificarea soldului curent pe o aplicație bancară de mobil.

- **REPEATABLE READ**

- **Când se folosește:** Când o tranzacție face calcule complexe pe anumite rânduri și necesită ca acele date să rămână "înghețate" până la finalizarea operațiunilor (menține lacătele de citire pe toată durata tranzacției). Previne Non-Repeatable Read și Lost Updates (prin generarea intenționată a unui Deadlock).
- **Scenariu din lumea reală:** Procesarea financiară de final de lună. Dacă un sistem calculează bonificații bazate pe venitul curent al unui angajat, acel venit nu are voie să fie modificat de un alt departament până nu se termină generarea fluturașului de salariu.

- **SERIALIZABLE**

- **Când se folosește:** Când este necesară consistență absolută. Pune lacăte pe întregi intervale de căutare (Range Locks), interzicând inclusiv inserările

noi (previne Phantom Reads). Execută tranzațiile ca și cum ar fi rulate secvențial.

- **Scenariu din lumea reală:** Sisteme de rezervări critice, cum ar fi cumpărarea de bilete de avion sau la un concert. Trebuie garantat sută la sută că două persoane nu pot rezerva scaunul 14A simultan și că sistemul raportează exact locurile disponibile.

### 3. Compromisuri între performanță și consistența datelor

Lucrul cu bazele de date presupune un compromis permanent (trade-off) între **Performanță (Concurență)** și **Consistență (Izolare)**.

- Dacă optăm pentru **izolare ridicată** (ex: `SERIALIZABLE`), garantăm că datele sunt 100% corecte și nu apar anomalii. Totuși, performanța scade drastic, deoarece tranzațiile stau blocate la "cozi" de așteptare pentru a primi lacăte exclusive. Într-o aplicație cu mii de utilizatori, acest lucru ar duce la timeout-uri frecvente.
- Dacă optăm pentru **performanță maximă** (ex: `READ UNCOMMITTED`), aplicația va răspunde instantaneu și resursele vor fi mereu disponibile, dar riscăm să luăm decizii de business bazate pe date invalide sau fanteziste (Dirty Reads, Lost Updates).
- Arhitectura corectă nu înseamnă setarea unui singur nivel pentru toată aplicația, ci adaptarea acestuia per tranzație: interogările critice folosesc izolare strictă, iar cele informative folosesc izolare relaxată.

### 4. Analiza performanței inserărilor în lot cu concluzii

Pentru a analiza impactul arhitecturii asupra performanței, am realizat un test de stres: inserarea a 5000 de înregistrări folosind trei abordări diferite.

*Vezi graficul din Excel*

#### Concluzii tehnice asupra performanței:

1. **Abordarea Auto-commit (~3532 ms):** A generat de departe cel mai slab timp. Explicația constă în overhead-ul uriaș de rețea și I/O pe disc. Pentru fiecare rând inserat, baza de date deschide o tranzație, scrie informația pe disc (în *Transaction Log* pentru a asigura durabilitatea) și o închide, de 5000 de ori consecutiv.

2. **Tranzacție Unică (~144 ms):** Cea mai rapidă abordare în testul curent. Prin gruparea tuturor operațiunilor într-o singură tranzacție, SGBD-ul menține datele în memoria RAM și face o singură scriere costisitoare pe disc la momentul apelului COMMIT.
3. **Commit în loturi (Batch de 100) (~250 ms):** Deși este ușor mai lentă decât tranzacția unică, aceasta este **soluția optimă recomandată**. Într-un scenariu real cu milioane de rânduri, o "Tranzacție Unică" ar consuma integral memoria RAM a serverului și ar bloca tabela o perioadă lungă. "Batch Inserts" reprezintă un compromis ideal: descarcă memoria periodic (la fiecare 100 de rânduri) și salvează pe disc eficient. Menține un timp excelent de execuție, protejând în același timp resursele sistemului.